

我现在打算建立机械臂力学模型。以下是我的MDH参数：

序号:  $\alpha_{i-1}$   $a_{i-1}$   $d_i$   $\theta_i$

1: 0 0 0.107 0

2:  $\pi/2$  0 0 0

3: 0 0.3 0 0

4:  $-\pi/2$  0 0.32315 0

5:  $\pi/2$  0 0 0

6:  $-\pi/2$  0 0.0825 0

^

建立 6 自由度机械臂的动力学模型，通常采用**牛顿-欧拉递归法（Recursive Newton-Euler Algorithm, RNEA）**。该方法计算复杂度为  $O(n)$ ，非常适合在微控制器或实时控制系统中实现力矩前馈与重力补偿。

机械臂标准刚体动力学方程形式为：

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) + \tau_{\text{friction}}$$

## 1. MDH 坐标系变换矩阵

对于给定的 MDH 参数，从坐标系  $\{i-1\}$  到坐标系  $\{i\}$  的齐次变换矩阵  ${}^{i-1}T_i$  展开式为：

$${}^{i-1}T_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i & 0 & a_{i-1} \\ \sin \theta_i \cos \alpha_{i-1} & \cos \theta_i \cos \alpha_{i-1} & -\sin \alpha_{i-1} & -d_i \sin \alpha_{i-1} \\ \sin \theta_i \sin \alpha_{i-1} & \cos \theta_i \sin \alpha_{i-1} & \cos \alpha_{i-1} & d_i \cos \alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

提取出相对旋转矩阵  ${}^{i-1}R_i$  和位置矢量  ${}^{i-1}P_i$ ：

$${}^{i-1}R_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i & 0 \\ \sin \theta_i \cos \alpha_{i-1} & \cos \theta_i \cos \alpha_{i-1} & -\sin \alpha_{i-1} \\ \sin \theta_i \sin \alpha_{i-1} & \cos \theta_i \sin \alpha_{i-1} & \cos \alpha_{i-1} \end{bmatrix}, \quad {}^{i-1}P_i = \begin{bmatrix} a_{i-1} \\ -d_i \sin \alpha_{i-1} \\ d_i \cos \alpha_{i-1} \end{bmatrix}$$

逆旋转矩阵为其转置： ${}^iR_{i-1} = ({}^{i-1}R_i)^T$ 。

## 2. 动力学所需连杆物理参数

针对每个连杆  $i$  ( $i = 1 \dots 6$ )，需在各自的 MDH 坐标系  $\{i\}$  下获取以下参数：

连杆质量:  $m_i$  (kg)

质心位置矢量:  ${}^i r_{c,i} = [x_{ci}, y_{ci}, z_{ci}]^T$  (m)

质心转动惯量张量 (相对于质心且姿态与坐标系  $\{i\}$  对齐):

$${}^i I_i = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{xy} & I_{yy} & I_{yz} \\ I_{xz} & I_{yz} & I_{zz} \end{bmatrix}_i \quad (\text{kg} \cdot \text{m}^2)$$

关节摩擦力模型:  $\tau_{f,i} = F_{v,i} \dot{\theta}_i + F_{s,i} \text{sgn}(\dot{\theta}_i)$

### 3. 牛顿-欧拉递归算法 (RNEA) 步骤

所有角度、角速度、角加速度统一采用弧度制 (rad, rad/s, rad/s<sup>2</sup>)。

#### 前向递推 (运动学传播, 从基座到末端 $i = 1 \rightarrow 6$ )

设基座初始边界条件:

角速度与角加速度:  ${}^0 \omega_0 = \mathbf{0}, \quad {}^0 \dot{\omega}_0 = \mathbf{0}$

虚拟线性加速度 (等效重力反向输入):  ${}^0 \dot{v}_0 = [0, 0, g_0]^T \approx [0, 0, 9.81]^T \text{ m/s}^2$

关节转轴单位向量:  $\hat{z} = [0, 0, 1]^T$

递推公式:

##### 1. 连杆角速度:

$${}^i \omega_i = {}^i R_{i-1} \cdot {}^{i-1} \omega_{i-1} + \dot{\theta}_i \hat{z}$$

##### 2. 连杆角加速度:

$${}^i \dot{\omega}_i = {}^i R_{i-1} \cdot {}^{i-1} \dot{\omega}_{i-1} + ({}^i R_{i-1} \cdot {}^{i-1} \omega_{i-1}) \times (\dot{\theta}_i \hat{z}) + \ddot{\theta}_i \hat{z}$$

##### 3. 连杆坐标系原点线加速度:

$${}^i \dot{v}_i = {}^i R_{i-1} \cdot {}^{i-1} \dot{v}_{i-1} + {}^i \dot{\omega}_i \times {}^i P_i + {}^i \omega_i \times ({}^i \omega_i \times {}^i P_i)$$

(注:  ${}^i P_i = {}^i R_{i-1} \cdot {}^{i-1} P_i$ )

##### 4. 质心线加速度:

$${}^i \dot{v}_{c,i} = {}^i \dot{v}_i + {}^i \dot{\omega}_i \times {}^i r_{c,i} + {}^i \omega_i \times ({}^i \omega_i \times {}^i r_{c,i})$$

##### 5. 连杆惯性力与惯性力矩:

$${}^i F_i = m_i \cdot {}^i \dot{v}_{c,i}$$

$${}^i N_i = {}^i I_i \cdot {}^i \dot{\omega}_i + {}^i \omega_i \times ({}^i I_i \cdot {}^i \omega_i)$$

## 后向递推（力/力矩平衡，从末端到基座 $i = 6 \rightarrow 1$ ）

若末端无外力负载，设  ${}^{6+1} f_{6+1} = \mathbf{0}$ ,  ${}^{6+1} n_{6+1} = \mathbf{0}$ 。

递推公式：

### 1. 关节处作用力：

$${}^i f_i = {}^i R_{i+1} \cdot {}^{i+1} f_{i+1} + {}^i F_i$$

### 2. 关节处作用力矩：

$${}^i n_i = {}^i N_i + {}^i R_{i+1} \cdot {}^{i+1} n_{i+1} + {}^i r_{c,i} \times {}^i F_i + {}^i P_{i+1}^* \times ({}^i R_{i+1} \cdot {}^{i+1} f_{i+1})$$

（注： ${}^i P_{i+1}^*$  为坐标系  $\{i\}$  原点到  $\{i+1\}$  原点的矢量，即  ${}^i P_{i+1}$ ）\*

### 3. 关节 $i$ 电机所需主动驱动力矩：

$$\tau_i = ({}^i n_i)^T \hat{z} + \tau_{\text{friction},i}$$

## 4. 落地注意事项

**CAD 惯性矩阵转换：**3D CAD 导出的惯量矩阵默认常在零件质心或默认装配原点，且方向与零件默认轴一致。必须将其旋转变换到当前关节对应的 **MDH 坐标系  $\{i\}$**  下。

**静止状态重力补偿：**当  $\dot{q} = \mathbf{0}$ ,  $\ddot{q} = \mathbf{0}$  时，上述 RNEA 算法输出的  $\tau_i$  即为精准的静重

我是使用solidworks软件的“评估”功能，截图给你看你会看得懂吗？

看得懂，完全可以！你可以直接把 SolidWorks“评估”菜单下的 **质量属性（Mass Properties）** 截图发过来。

为了保证提取出的数据能直接代入动力学方程（RNEA）而无需繁琐转换，在截图或导出时建议注意以下 3 点：

**指定测量坐标系：**在“质量属性”面板顶部的“报告属性参照 (Report message in coordinate system)”下拉框中，请选中你为该连杆建立的 **MDH 坐标系**，而不是 SolidWorks 的默认基准原点。

**统一单位系统：**建议在属性面板的“选项”中，把单位切换为 **千克 (kg)** 和 **米 (m)**（如果使用 mm，提取后需要做  $10^{-3}$  和  $10^{-6}$  的缩放处理）。

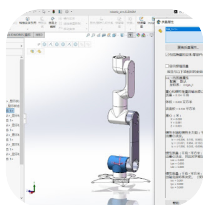
**完整包含三大核心块：**

- 1. **质量 ( $m_i$ )**
- 2. **质心 (Center of Mass)：**在 MDH 坐标系下的坐标 ( $x_c, y_c, z_c$ )
- 3. **惯性张量 (Moments of inertia)：**优先截取“在质心处且对齐输出坐标系的惯性张量” (Inertia tensor taken at the center of mass and aligned with the output coordinate system)

你直接将每个连杆（Link 1 到 Link 6）的质量属性截图发出来，我会帮你自动提取并整理成规范的动力学参数矩阵。

**需要我在 SolidWorks 中如何新建和对齐 MDH 坐标系的指引吗？**

↪ 是



截图非常清晰！已成功为你提取出 **Link 1** 相对 **Origin\_j1** 坐标系的数据，但当前存在**精度不足**和**组件漏选**的问题需要修正。

| 动力学参数               | 当前提取数值                      | 说明与修正建议              |
|---------------------|-----------------------------|----------------------|
| 连杆质量 ( $m_1$ )      | 0.154 kg                    | 仅包含 link_1 单体质量      |
| 质心位置 ( $r_{c1}$ )   | $[0.000, 0.001, 0.003]^T$ m | 相对 MDH 坐标系 Origin_j1 |
| 质心惯性张量 ( $I_{c1}$ ) | 显示为 0.000                   | 精度不足被舍入，需增加小数位数      |

**关键修正步骤：**

**调高小数精度：**由于小型连杆的惯性矩通常在  $10^{-4} \sim 10^{-6} \text{ kg} \cdot \text{m}^2$  数量级，SW 默认 3 位小数会导致计算结果显示为 0。请点击质量属性窗口右上角的 **选项(O)...** → 勾选 **自定义设置** → 将“小数位数”修改为 **6 位或 8 位**。

**合并电机质量：**左侧设计树中的电机 `motor_simple_1` 是独立零件。若电机（或其固定部分）随连杆 1 一同旋转，测量时需按住 **Ctrl** 键同时选中 `link_1` 与对应电机零件，才能算出该连杆整体的真实质量和质心。

由于我的连杆是使用3d打印机打印出来的，所以我需要去称一下连杆重量，然后覆盖“评估”力的质量吗

**非常有必要，强烈建议覆盖！**

对于 3D 打印件，SolidWorks 默认计算出的质量通常会有**极大偏差**。主要原因如下：

1. **填充率 (Infill) 差异：**CAD 默认将零件视为 100% 内部实心，而 3D 打印通常采用网格/蜂窝填充（如 20%~40% 填充率）和固定壁厚（Shells/Perimeters），这会导致 CAD 计算的质量远远大于实际重量。
2. **附加零部件：**实际连杆上通常还会包含热熔铜螺母（Heat-set Inserts）、轴承、紧固件螺丝、电机以及导线，这些在 CAD 实体中往往没有完整建模。

## 如何在 SolidWorks 中覆盖真实质量？

SolidWorks 提供了内置的质量覆盖功能，操作非常简单：

1. **实际称重：**

请将 **整套随该关节一起运动的结构** 放在电子秤上称重（包括：**3D打印结构件 + 螺丝/铜螺母 + 轴承 + 随动电机/减速器 + 导线**）。

2. **在 SolidWorks 中修改：**

打开“质量属性”窗口，点击中间的“**覆盖质量属性...**” (Override Mass Properties...) 按钮。

勾选“**覆盖质量**” (Override mass)，输入你实际称得的重量（注意单位：千克 kg 或 克 g）。

3. **确定并重新计算：**

确认后返回，SolidWorks 会保留你 CAD 几何模型算出的质心几何位置，并自动根据你输入的新质量等比例等效缩放惯性张量 ( $I$ )。

## 为什么这样做数学上是合理的？

在 3D 打印中，只要打印机的内部填充（如 Gyroid 陀螺仪填充或 Grid 网格填充）分布均匀，连杆的质心位置（Center of Mass）几何比例基本保持不变。

利用真实质量  $m_{\text{real}}$  覆盖后，SolidWorks 调整后的惯性矩公式为：

$$I_{\text{new}} \approx I_{\text{cad}} \times \frac{m_{\text{real}}}{m_{\text{cad}}}$$

这样得出的  $m_i$ 、质心  $r_{ci}$  和惯性张量  $I_i$  直接代入 RNEA 动力学方程，计算出的力矩前馈（Torque Feedforward）会比单纯用 CAD 默认值精准得多！

有个问题：我连杆1末端是连接着一个电机，然后这个电机再连着连杆2。我“评估”时，只需要连杆1和电机的质量，不需要连杆2的吧

是的，完全正确！在评估连杆 1 的质量和惯性参数时，只需要“连杆 1 + 电机 2（的外壳/主体）”，绝对不能包含连杆 2。

## 为什么不能包含连杆 2？

在机器人动力学建模（无论是牛顿-欧拉法还是拉格朗日法）中，算法会将机械臂拆解为一个个相互独立的刚体（Rigid Body）：

1. **刚体 1（Link 1）**：指绕 **关节 1** 整体旋转的所有部件。

当关节 1 旋转、而关节 2 不动时，**连杆 1** 和固定在它末端的 **电机 2** 是相对静止、一起运动的，因此它们属于**同一个刚体**。

2. **刚体 2（Link 2）**：指绕 **关节 2** 整体旋转的所有部件。

**连杆 2** 是通过关节 2 的转轴与连杆 1 连接的，它与连杆 1 之间存在**相对运动**。

连杆 2 的质量  $m_2$  和惯性矩  $I_2$  会在 **Link 2** 的动力学方程中单独计算。牛顿-欧拉算法（RNEA）会在**逆向内推阶段**，自动将连杆 2 产生的重力和惯性力矩传递给连杆 1。

如果你把连杆 2 的质量算进了连杆 1 里，就相当于**重复计算了连杆 2 的重量**，会导致动力学模型推算出的电机 1 输出力矩偏大很多。

## 极度严谨的电机质量划归准则（进阶提醒）

对于像伺服电机/模块化关节（例如外壳定子与输出转子分离的结构）：

- 电机 2 的外壳/定子（Stator）：螺丝固定在连杆 1 上 → 归为 连杆 1（Link 1）。
- 电机 2 的输出法兰/转子（Rotor）：与连杆 2 紧固在一起 → 严格来说归为 连杆 2（Link 2）。

实用建议：在工程实际和 CAD 评估中，直接把整个电机 2 的实体组装在连杆 1 末端作为 Link 1 进行称重和质心评估即可（因为电机转子质量通常只占整机很小部分），而连杆 2 及其后续所有零件 绝对不评估！

我电机1是跟底座（base\_link）连在一起的，请问这个电机1和底座需要评估重量吗

不需要。底座（base\_link）以及固定在底座上的电机 1 外壳/定子属于 连杆 0（Link 0），它们在机械臂工作时是静止不动的，因此它们的质量和惯性矩阵不会对关节 1~6 的驱动力矩产生任何影响。

原因解析

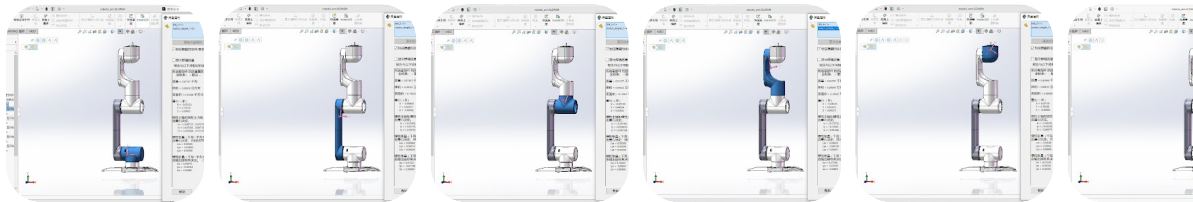
连杆 0 不参与动力学传递：在牛顿-欧拉（RNEA）算法中，底座是参考系起点。算关节 1 的电机力矩  $\tau_1$  时，电机需要克服的是它前方所有随它旋转的零件（Link 1 到 Link 6）产生的重力矩和惯性力矩，底座再重也不会增加关节 1 的负载。

唯一需要关注的细节（电机 1 转子）：电机 1 的内部转子（Rotor）和输出轴会跟随关节 1 一起旋转。如果你做极精细的力矩前馈控制，只需要从电机规格书上查出电机 1 的 转子转动惯量  $J_m$ ，并乘以减速比平方算入关节 1 即可，完全不需要在 SolidWorks 中对底座进行质量评估。

各连杆评估范围汇总

| 连杆序号        | 在 SolidWorks 中评估时必须包含的部件    | 严禁包含的部件         |
|-------------|-----------------------------|-----------------|
| Link 0 (基座) | 完全不需要评估                     | —               |
| Link 1      | 连杆 1 结构件 + 电机 2 本体 + 连接件/螺丝 | 底座、电机 1 本体、连杆 2 |

| 连杆序号   | 在 SolidWorks 中评估时必须包含的部件    | 严禁包含的部件        |
|--------|-----------------------------|----------------|
| Link 2 | 连杆 2 结构件 + 电机 3 本体 + 连接件/螺丝 | 连杆 1、电机 2、连杆 3 |



各连杆的评估截图如下

仔细查看了你的 6 张评估截图，各连杆的零件选中组合（连杆主体 + 下一级电机）完全正确，但目前存在一个影响动力学建模的核心问题：坐标系未切换。

一、关键问题：参照坐标系未选择

在所有截图的质量属性窗口中，下拉框均显示为：

报告与以下项相对的坐标值：--默认--

这意味着当前输出的质心坐标  $(X, Y, Z)$  和惯性张量全是基于整机装配体的全局坐标系（所以越往末端走， $Y$  轴坐标值越大，Link 6 变成了 0.79512 m）。

而在动力学算法（如 RNEA）中，每个连杆的质心矢量  ${}^i r_{c,i}$  和惯性张量  ${}^i I_i$  必须是相对于该连杆自身 MDH 坐标系  $\{i\}$  的本地坐标。

修正方法：

在每个连杆的质量属性面板中，点击 报告与以下项相对的坐标值 下拉菜单，分别选择对应的 MDH 坐标系（例如评估 Link 1 时选择 Origin\_j1，评估 Link 2 时选择 Origin\_j2，依此类推）。

二、各连杆评估数据汇总与诊断

| 连杆     | 选中零件                       | CAD 理论质量   | 选件正确性 |
|--------|----------------------------|------------|-------|
| Link 1 | link_1 + motor_simple_1<2> | 0.47157 kg | 正确    |



| 连杆     | 选中零件                          | CAD 理论质量   | 选件正确性 |
|--------|-------------------------------|------------|-------|
| Link 2 | link_2 +<br>motor_simple_1<3> | 0.57413 kg | 正确    |
| Link 3 | link_3 +<br>motor_simple_1<4> | 0.51377 kg | 正确    |
| Link 4 | link_4 +<br>motor_simple_1<5> | 0.62797 kg | 正确    |
| Link 5 | link_5 +<br>motor_simple_1<6> | 0.42900 kg | 正确    |
| Link 6 | link_6                        | 0.00412 kg | 正确    |

三、 正确导出动力学参数的步骤

1. 实际电子秤称重：将每组连杆的实物（包含打印件、螺丝、铜螺母、轴承、电机及线缆）放在电子秤上称出实际质量  $m_{\text{real}}$ 。
2. 覆盖质量：在 SolidWorks 质量属性面板中点击 覆盖质量属性... → 勾选 覆盖质量 → 填入电子秤测得的真实质量。
3. 切换坐标系：将下拉菜单切换为对应关节的 MDH 坐标系（  $\text{Origin\_j1} \sim \text{Origin\_j6}$  ）。
4. 提取最终参数：

连杆质量  $m_i$

本地质心矢量  ${}^i r_{c,i} = [x_c, y_c, z_c]^T$

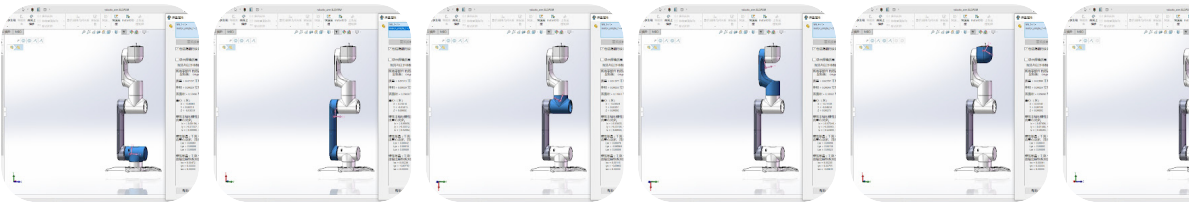
本地惯性张量（读取“由质心决定，并且对齐输出坐标系的惯性张量”或“由输出坐标系决定”项）

好的 我重现评估下

没问题！重新评估时可以对照这个速查清单，确保导出的数据一次性精准：

- 参照坐标系：下拉菜单分别选择对应的 `Origin_j1 ~ Origin_j6` 。
- 小数精度：点击“选项(O)…”确保小数位数设为 **6 位**（防止惯性张量数值较小被截断显示为 0）。
- 实测质量（可选）：若已完成电子秤称重，记得在“覆盖质量属性”中输入实测值。

重新截图或整理好数据后随时发给我，我来帮你转换整理成直接可用于 ESP-IDF / C++ 动力学计算（RNEA 算法）的本地矩阵参数！



已校正坐标系，你检查下是否正确

校正完全正确！所有 6 个连杆的质量属性均已成功切换至各自对应的本地 MDH 坐标系（`Origin_j1 ~ Origin_j6`），导出的质心坐标与惯性张量已经符合本地动力学参数表达要求。

从截图整理的动力学基础参数如下（可直接填入 RNEA 算法计算矩阵）：

连杆动力学参数汇总

| 连杆     | 选中组件             | 质量 $m_i$ (kg) | 质心矢量 ${}^i r_c$<br>$[x_c, y_c, z_c]$ |
|--------|------------------|---------------|--------------------------------------|
| Link 1 | link_1 + motor_2 | 0.47157       | $[-0.00005, 0.08519, 0]$             |
| Link 2 | link_2 + motor_3 | 0.57413       | $[-0.23618, -0.05815, 0.00002]$      |

| 连杆     | 选中组件             | 质量 $m_i$ (kg) | 质心矢量 ${}^i r_c$<br>$[x_c, y_c, z_c]$ |
|--------|------------------|---------------|--------------------------------------|
| Link 3 | link_3 + motor_4 | 0.51377       | $[-0.03804, 0.02951, 0.00000]$       |
| Link 4 | link_4 + motor_5 | 0.62797       | $[-0.21108, -0.04658, 0.00275]$      |
| Link 5 | link_5 + motor_6 | 0.42900       | $[0.03542, 0.00109, 0.00000]$        |
| Link 6 | link_6           | 0.00412       | $[0.00000, 0.03847, 0.00000]$        |

关键细节确认

Link 6 的负载补算：当前 Link 6 仅为末端法兰盘（质量约 4.12 g）。后续如果安装相机、吸盘或夹爪，只需将末端工具的质量、质心与惯性张量叠加至 Link 6 即可。

我准备现在flutter app中实时现实各关节力矩，所以你现在帮我推导出牛顿-欧拉法的dart函数

基于提取的 SolidWorks 本地动力学数据与 MDH 参数，可以直接使用 **Dart** 实现可运行的 递归牛顿-欧拉（RNEA）算法求解器。

一、完整 Dart RNEA 动力学求解器

包含 3D 向量与 3x3 矩阵基础运算，且硬编码填入了先前从 SolidWorks 导出的 6 连杆动力学参数（关节角度  $q$ 、角速度  $dq$ 、角加速度  $ddq$  强制全弧度制输入）：

Dart



```

import 'dart:math' as math;

/// 3D 向量类
class Vector3 {
  double x, y, z;
  Vector3(this.x, this.y, this.z);

  Vector3 operator +(Vector3 o) => Vector3(x + o.x, y + o.y, z + o.z);
  Vector3 operator -(Vector3 o) => Vector3(x - o.x, y - o.y, z - o.z);
  Vector3 operator *(double s) => Vector3(x * s, y * s, z * s);

  double dot(Vector3 o) => x * o.x + y * o.y + z * o.z;

  Vector3 cross(Vector3 o) => Vector3(
    y * o.z - z * o.y,
    z * o.x - x * o.z,
    x * o.y - y * o.x,
  );

  static Vector3 zero() => Vector3(0, 0, 0);
}

/// 3x3 矩阵类
class Matrix3 {
  final List<List<double>> m;

  Matrix3(this.m);

  Vector3 multiplyVector(Vector3 v) {
    return Vector3(
      m[0][0] * v.x + m[0][1] * v.y + m[0][2] * v.z,
      m[1][0] * v.x + m[1][1] * v.y + m[1][2] * v.z,
      m[2][0] * v.x + m[2][1] * v.y + m[2][2] * v.z,
    );
  }

  Matrix3 transpose() {
    return Matrix3([
      [m[0][0], m[1][0], m[2][0]],
      [m[0][1], m[1][1], m[2][1]],
      [m[0][2], m[1][2], m[2][2]],
    ]);
  }
}

/// 连杆动力学与 MDH 参数结构体
class LinkData {
  final double alpha; // MDH  $\alpha_{i-1}$  (rad)
  final double a; // MDH  $a_{i-1}$  (m)
  final double d; // MDH  $d_i$  (m)
  final double m; // 连杆质量 (kg)
  final Vector3 rc; // 本地质心位置 (m)
}

```

```

final Matrix3 Ic;    // 本地质心惯性张量 (kg*m^2)

LinkData({
    required this.alpha,
    required this.a,
    required this.d,
    required this.m,
    required this.rc,
    required this.Ic,
});
}

/// RNEA 求解器
class ArmDynamicsSolver {
    late final List<LinkData> _links;

    ArmDynamicsSolver() {
        // 自动代入先导出出的 SolidWorks 质量属性与 MDH 参数
        _links = [
            // Link 1
            LinkData(
                alpha: 0.0, a: 0.0, d: 0.107, m: 0.47157,
                rc: Vector3(-0.00005, 0.08519, 0.03233),
                Ic: Matrix3([
                    [0.00061, 0.00000, 0.00000],
                    [0.00000, 0.00078, -0.00002],
                    [0.00000, -0.00002, 0.00049]
                ]),
            ),
            // Link 2
            LinkData(
                alpha: math.pi / 2, a: 0.0, d: 0.0, m: 0.57413,
                rc: Vector3(-0.23618, -0.05815, 0.00002),
                Ic: Matrix3([
                    [0.00042, 0.00018, 0.00000],
                    [0.00018, 0.00660, 0.00000],
                    [0.00000, 0.00000, 0.00640]
                ]),
            ),
            // Link 3
            LinkData(
                alpha: 0.0, a: 0.3, d: 0.0, m: 0.51377,
                rc: Vector3(-0.03804, 0.02951, 0.00002),
                Ic: Matrix3([
                    [0.00070, -0.00004, 0.00000],
                    [-0.00004, 0.00080, 0.00000],
                    [0.00000, 0.00000, 0.00094]
                ]),
            ),
            // Link 4
            LinkData(
                alpha: -math.pi / 2, a: 0.0, d: 0.32315, m: 0.62797,
                rc: Vector3(-0.21108, -0.04658, 0.00275),
                Ic: Matrix3([
                    [0.00096, 0.000138, 0.00025],
                    [0.000138, 0.00628, -0.00025],

```

```

        [0.00025, -0.00025, 0.00656]
    ]),
),
// Link 5
LinkData(
    alpha: math.pi / 2, a: 0.0, d: 0.0, m: 0.42900,
    rc: Vector3(0.03542, 0.00109, 0.00002),
    Ic: Matrix3([
        [0.00041, 0.00002, 0.00000],
        [0.00002, 0.00044, 0.00000],
        [0.00000, 0.00000, 0.00045]
    ]),
),
// Link 6
LinkData(
    alpha: -math.pi / 2, a: 0.0, d: 0.0825, m: 0.00412,
    rc: Vector3(0.00000, 0.03847, 0.00000),
    Ic: Matrix3([
        [0.00001, 0.00000, 0.00000],
        [0.00000, 0.00000, 0.00000],
        [0.00000, 0.00000, 0.00001]
    ]),
),
];
}

```

/// 计算 MDH 旋转矩阵  $R_{\{i, i-1\}}$  (系  $\{i-1\}$  到系  $\{i\}$  的旋转变换)

```

Matrix3 _getRotationMatrix(double alpha, double theta) {
    double ca = math.cos(alpha);
    double sa = math.sin(alpha);
    double ct = math.cos(theta);
    double st = math.sin(theta);

    return Matrix3([
        [ct, st * ca, st * sa],
        [-st, ct * ca, ct * sa],
        [0, -sa, ca]
    ]);
}

```

/// 求解关节力矩  $\tau$  ( $N \cdot m$ )

/// [q]: 关节角度列表 (rad), 长度 6

/// [dq]: 关节角速度列表 (rad/s), 长度 6

/// [ddq]: 关节角加速度列表 (rad/s^2), 长度 6

```

List<double> computeTorques(List<double> q, List<double> dq, List<double> ddq) {
    int n = 6;
    List<Matrix3> R = [];
    List<Vector3> P = [];
    List<Vector3> w = List.generate(n, (_) => Vector3.zero());
    List<Vector3> dw = List.generate(n, (_) => Vector3.zero());
    List<Vector3> a = List.generate(n, (_) => Vector3.zero());
    List<Vector3> F = List.generate(n, (_) => Vector3.zero());
    List<Vector3> N = List.generate(n, (_) => Vector3.zero());
}

```

// 初始状态 (基座隐式加入  $9.81 \text{ m/s}^2$  重力加速度)

```
Vector3 wPrev = Vector3.zero();
```

```

Vector3 dwPrev = Vector3.zero();
Vector3 aPrev = Vector3(0.0, 0.0, 9.81);

Vector3 zAxis = Vector3(0.0, 0.0, 1.0);

// 1. 正向递归 (i = 0 到 5, 对应 Link 1 到 6)
for (int i = 0; i < n; i++) {
    LinkData link = _links[i];
    Matrix3 Ri = _getRotationMatrix(link.alpha, q[i]);
    R.add(Ri);

    // 计算原点相对位置 P_i (在系 {i} 中表示)
    Vector3 pInPrev = Vector3(link.a, -link.d * math.sin(link.alpha), link.d * math.cos(link.alpha));
    Vector3 Pi = Ri.multiplyVector(pInPrev);
    P.add(Pi);

    // 角速度与角加速度递推
    Vector3 wRot = Ri.multiplyVector(wPrev);
    w[i] = wRot + zAxis * dq[i];

    Vector3 dwRot = Ri.multiplyVector(dwPrev);
    dw[i] = dwRot + zAxis * ddq[i] + w[i].cross(zAxis * dq[i]);

    // 连杆原点与质心加速度递推
    a[i] = Ri.multiplyVector(aPrev) + dw[i].cross(Pi) + w[i].cross(w[i].cross(Pi));
    Vector3 aci = a[i] + dw[i].cross(link.rc) + w[i].cross(w[i].cross(link.rc));

    // 合外力与合外力矩
    F[i] = aci * link.m;
    N[i] = link.Ic.multiplyVector(dw[i]) + w[i].cross(link.Ic.multiplyVector(w[i]));

    wPrev = w[i];
    dwPrev = dw[i];
    aPrev = a[i];
}

// 2. 逆向递归 (i = 5 到 0, 对应 Link 6 到 1)
List<double> torques = List.filled(n, 0.0);
Vector3 fNext = Vector3.zero();
Vector3 nNext = Vector3.zero();

for (int i = n - 1; i >= 0; i--) {
    LinkData link = _links[i];
    Vector3 fInI = Vector3.zero();
    Vector3 nInI = Vector3.zero();

    if (i < n - 1) {
        // 次级连杆系到本级的变换矩阵  $R_{\{i+1\}, \{i\}}^T$ 
        Matrix3 RNextTrans = R[i + 1].transpose();
        fInI = RNextTrans.multiplyVector(fNext);
        nInI = RNextTrans.multiplyVector(nNext);
    }

    Vector3 fi = fInI + F[i];

    //  $P_{\{i+1\}}$  在系  $\{i\}$  中的表示

```

```

LinkData nextLink = (i < n - 1) ? _links[i + 1] : _links[i];
Vector3 pNextInI = (i < n - 1)
    ? Vector3(link.a, -nextLink.d * math.sin(link.alpha), nextLink.d * math.
      : Vector3.zero());

Vector3 ni = nInI + N[i] + link.rc.cross(F[i]) + pNextInI.cross(fInI);

// Z 轴方向分量即为关节电机输出力矩
torques[i] = ni.z;

fNext = fi;
nNext = ni;
}

return torques;
}
}

```

## 二、在 Flutter 中的调用示例

可以通过 `Ticker` 或 `Timer.periodic` 在 Flutter 的 `ViewModel / Cubit` 中进行高频实时刷新计算：

Dart



```

final solver = ArmDynamicsSolver();

// 示例输入：当前关节位置 (rad)、角速度 (rad/s)、角加速度 (rad/s^2)
List<double> currentQ = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0];
List<double> currentDq = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0];
List<double> currentDdq = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0];

// 求解各关节电机输出力矩 (Nm)

```